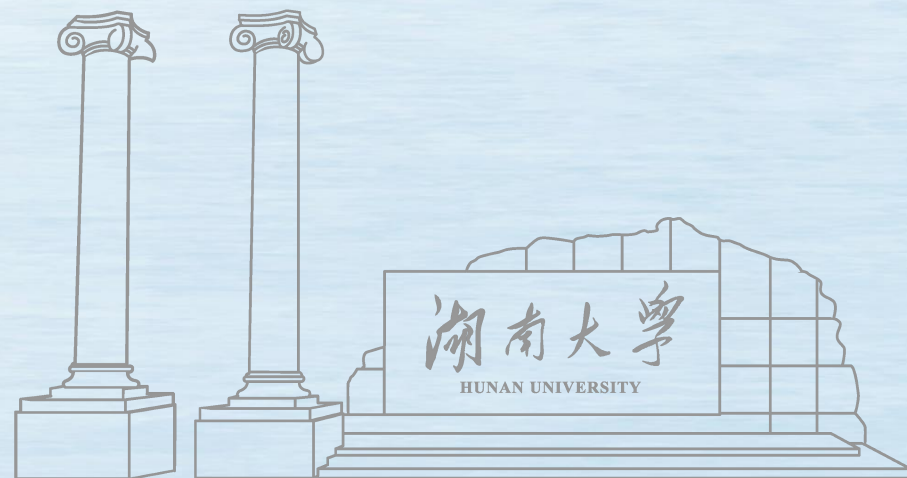


Splay Tree

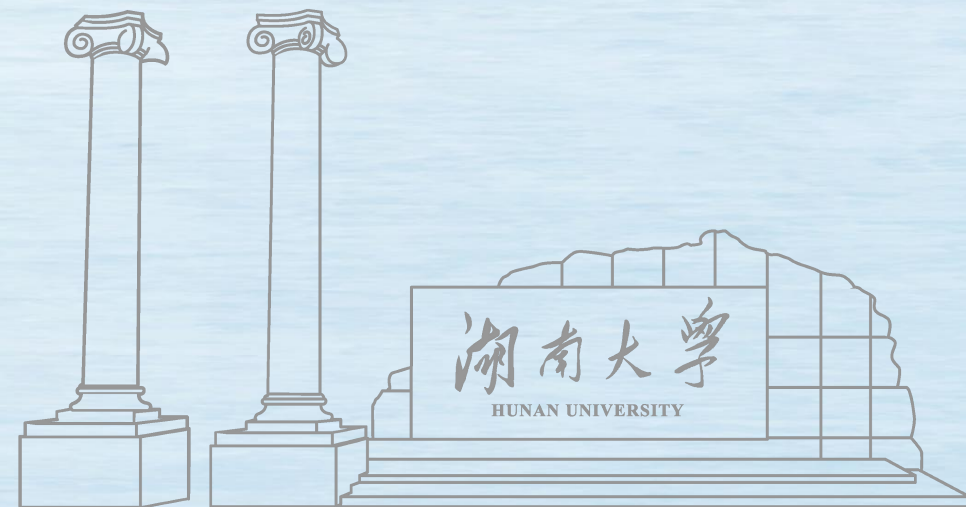
伸展树

第二次小班课个人资料



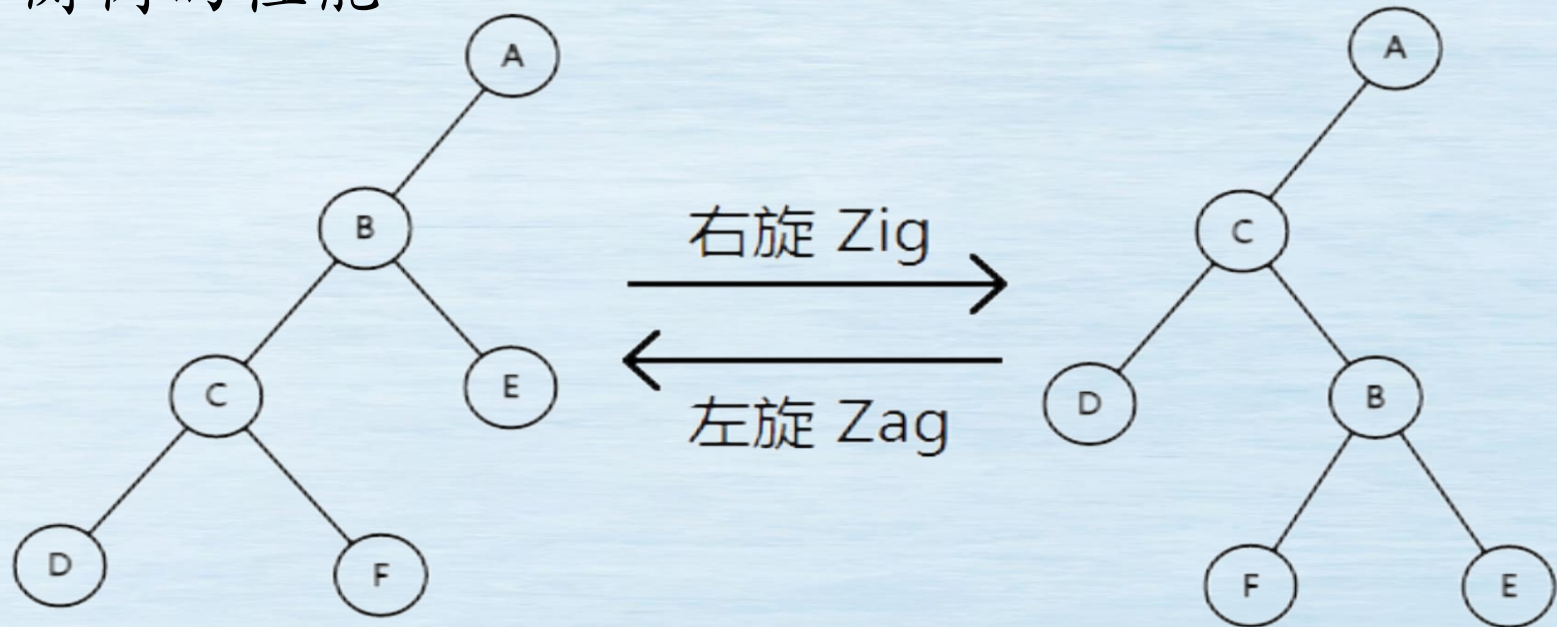
目录

- 1 定义
- 2 存储实现
- 3 调整方法
- 4 具体操作
- 5 时间复杂度
- 6 应用



一. 定义

伸展树是一种自调整的二叉搜索树，它通过一种称为“伸展”（splaying）的操作进行树形结构的重组。每次对树进行访问时，都会执行“伸展”操作，即通过一系列旋转将指定的节点移动到树的根部。这种自调整的策略旨在通过频繁访问的节点靠近根部，优化未来的操作效率。伸展树的一个显著特点是它没有显式的平衡约束，而是通过调整树结构以适应访问模式，从而在长期的操作序列中实现类似平衡树的性能^[1]



二. 存储实现

1. 伸展树结构定义

```
typedef struct SplayNode *Tree;
typedef int ElementType;
struct SplayNode
{
    Tree parent; //该结点的父节点，方便操作
    ElementType val; //结点值
    Tree lchild;
    Tree rchild;
    SplayNode(int val=0) //默认构造函数
    {
        parent=NULL;
        lchild=rchild=NULL;
        this->val=val;
    }
};
```


二. 存储实现

2. 存储结构

与普通的二叉查找树类似，伸展树的每个节点通常包含以下信息：

值 (Value)： 节点存储的数据元素。

左子节点指针 (Left Child)： 指向左子节点的指针。

右子节点指针 (Right Child)： 指向右子节点的指针。

父节点指针 (Parent)： 指向父节点的指针。

3. 实现方式

伸展树的存储实现通常采用指针式存储结构，即每个节点包含指向其父节点、左子节点和右子节点的指针。这种结构使得在进行旋转操作时，能够方便地调整节点之间的关系。

三. 调整方法

1. zig型

- 定义

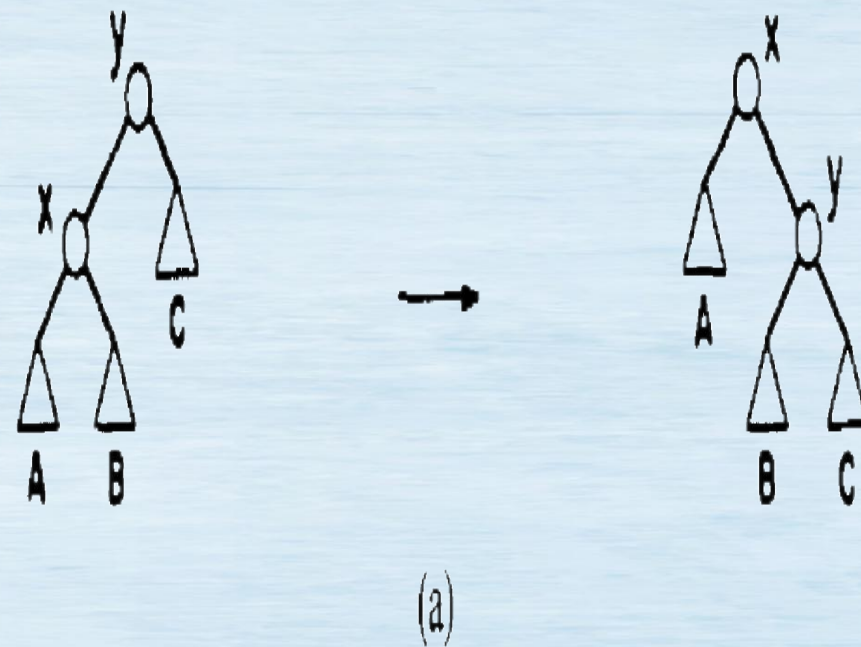
Zig 是伸展树中的基本旋转操作。当访问的节点是树的直接子节点且父节点是根节点时，会执行 Zig 操作。即，目标节点与其父节点之间形成的路径长度为1（只有一条边）。

- 操作

当目标节点是父节点的子节点（不管是左子节点还是右子节点），且父节点是树的根节点时，执行一次单旋转（rotate）操作，将目标节点移至树的根部。

- 例子

假设目标节点是 x ，其父节点是 y ，且 y 是根节点，进行一次旋转使得 x 成为根节点



三. 调整方法

2. zig-zig型

- 定义

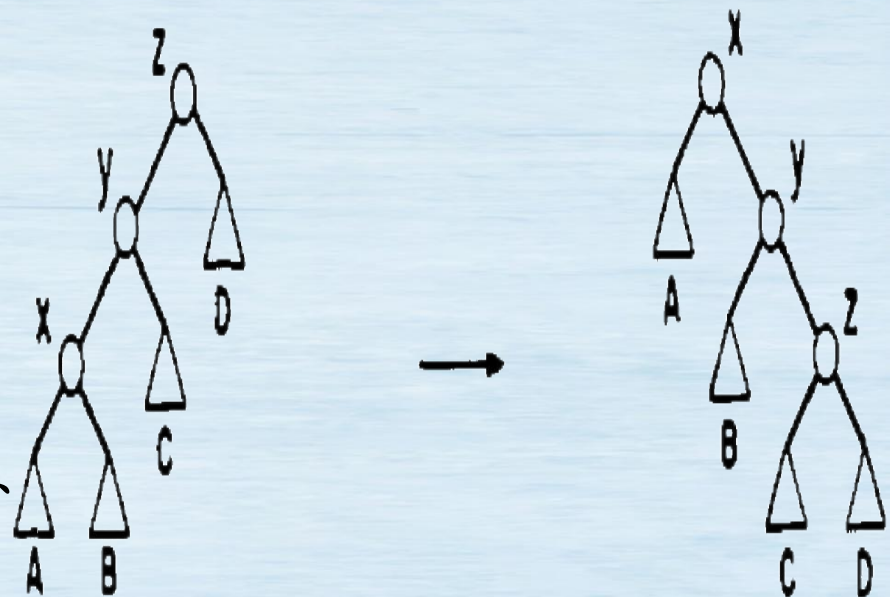
Zig-zig 是伸展树的旋转操作，当访问的节点与其父节点、祖父节点之间的关系满足两者都是同一方向（即目标节点和父节点要么都是左子节点，要么都是右子节点）时，会执行 Zig-zig 操作。

- 操作

在 Zig-zig 操作中，首先对父节点和祖父节点进行旋转，使得父节点和祖父节点的关系发生改变，接着再对目标节点和父节点进行旋转。最终目标节点将上升至树根。

- 例子

假设目标节点是 x ，父节点是 y ，祖父节点是 z ，且 x 和 y 之间都是左子节点， y 和 z 之间也是左子节点，执行两次旋转后， x 变成根节点。



三. 调整方法

3. zig-zag型

- 定义

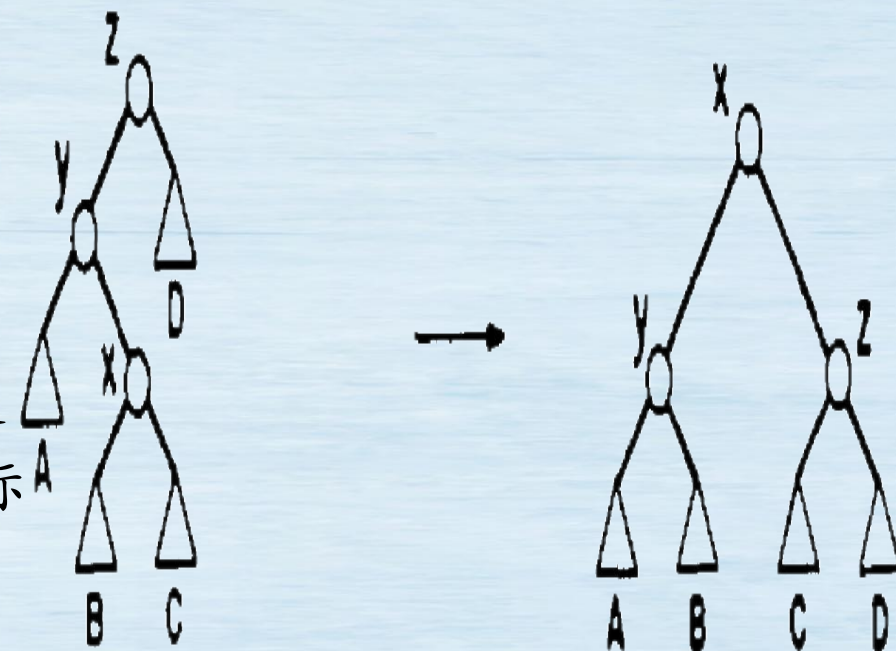
Zig-zag 是伸展树中的另一种旋转操作，当访问的节点与其父节点和祖父节点之间的关系满足节点和父节点方向相反（即目标节点是父节点的左子节点，父节点是祖父节点的右子节点，或者目标节点是父节点的右子节点，父节点是祖父节点的左子节点）时，会执行 Zig-zag 操作。

- 操作

在 Zig-zag 操作中，首先对目标节点和父节点进行一次旋转，然后对目标节点和祖父节点进行一次旋转。这样，目标节点会被“拉到”根部。

- 例子

假设目标节点是 x ，父节点是 y ，祖父节点是 z ，且 x 是 y 的右子节点， y 是 z 的左子节点，执行两次旋转后， x 成为根节点。



四. 具体操作

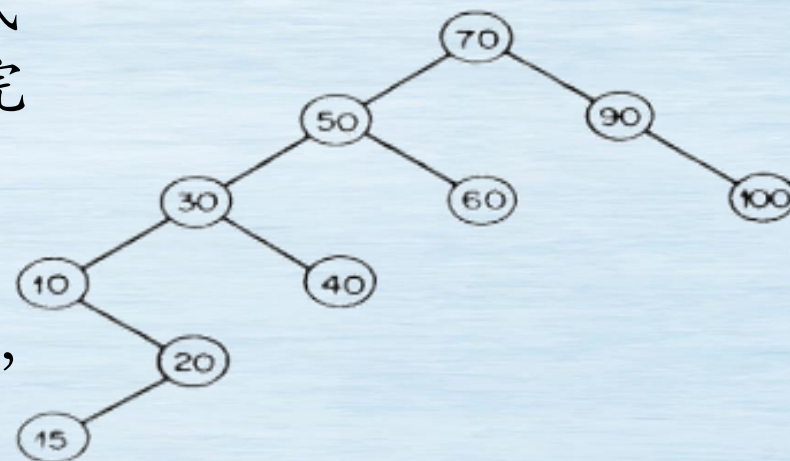
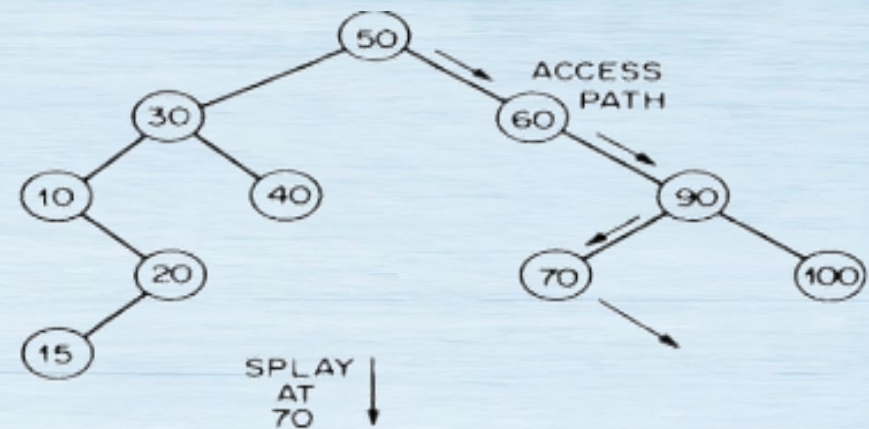
1. access(i, t)

- 定义

如果 i 在树 t 中，返回一个指向 i 位置的指针；否则，返回一个指向空节点的指针

- 操作

我们从树 t 的根开始，按照二叉搜索树的查找方式向下查找；如果当前节点 x 包含待查目标 i ，则算法完成，我们对 x 进行splaying操作并返回指向 x 的指针。如果搜索到达了空节点（即待查目标不存在），那么我们对搜索路径的最后一个节点进行splaying操作，并返回一个空指针。如果树为空，则取消splaying操作



四. 具体操作

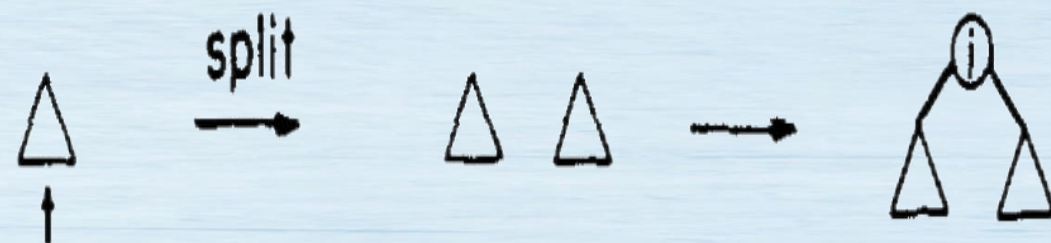
2. insert(i, t)

- 定义

向树 t 中插入一个元素 i , 假设此前 i 不存在。

- 操作

插入操作涉及先对树进行拆分, 将树分成两个子树。然后创建一个新的节点, 并将该节点作为树的根节点, 同时将两个子树作为它的左右子树



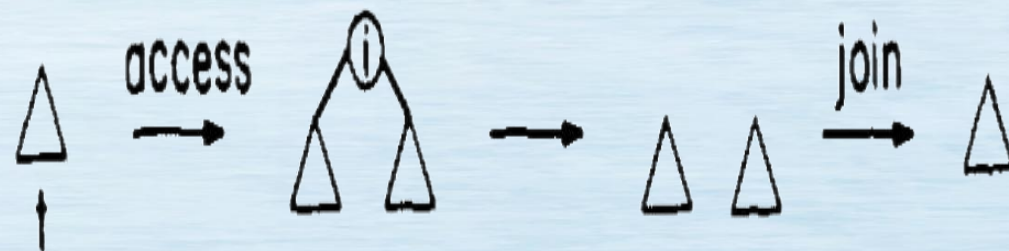
3. delete(i, t)

- 定义

从树 t 中删除元素 i , 假设 i 存在

- 操作

删除操作首先执行查找操作, 找到需要删除的节点。接着, 删除节点的左右子树会被合并, 并且树会在删除节点的父节点处进行伸展。

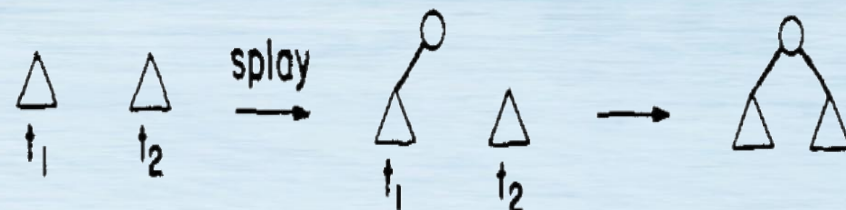


四. 具体操作

4. join(t1, t2)

- 定义

将树t1和t2合并成一棵新的树并返回新树的树根。该操作假设t1中所有元素都小于t2中的元素。合并后删除t1和t2。



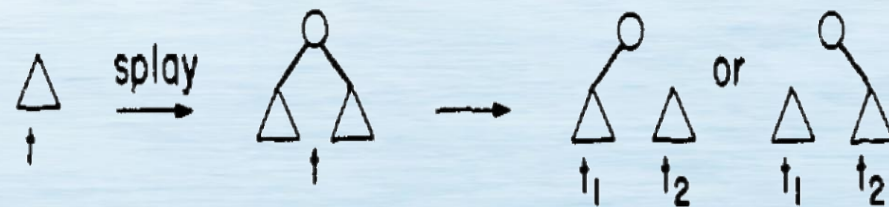
- 操作

合并操作首先访问第一个树中的最大节点，并将其伸展至根节点。然后，将第二棵树作为第一个树根节点的右子树附加到第一棵树上。

5. split(i, t):

- 定义

将t分成两棵子树t1和t2，所有小于等于i的元素在t1，所有大于i的元素在t2。然后删除树t。



- 操作

删除操作首先执行查找操作，找到需要删除的节点。接着，删除节点的左右子树会被合并，并且树会在删除节点的父节点处进行伸展。

五. 时间复杂度

伸展树 (Splay Tree) 是一种自平衡的二叉查找树 (BST)，其主要特点是在每次操作 (如查找、插入、删除) 后，将被访问的节点通过一系列旋转操作移动到树的根部。这种自我调整的机制使得伸展树在多次访问相同节点时表现出良好的平均性能。

伸展树的时间复杂度分析主要基于摊还分析 (Amortized Analysis)，即对一系列操作的平均性能进行评估。虽然单次操作的最坏时间复杂度可能较高，但通过摊还分析，可以证明伸展树在多次操作后的平均时间复杂度具有对数级别的增长

单次操作的最坏情况：在最坏情况下，伸展树的操作时间复杂度可能达到 $O(n)$ ，例如当树退化成链表时。

摊还时间复杂度：通过对一系列操作进行摊还分析，可以证明，伸展树的每次操作的平均时间复杂度为 $O(\log n)$ ，其中 n 是树中节点的数量。^[1]

六. 应用

主要应用场景

- 频繁访问相同元素的场景： 当某些元素被频繁访问时，伸展树能够通过将这些元素移至根部，减少未来访问这些元素的时间。
- 需要动态调整的数据集合： 伸展树在插入和删除操作后会自动调整结构，适用于元素频繁变化的应用场景。
- 内存受限的环境： 由于伸展树不需要额外的存储空间来维护平衡信息，其空间开销较小，适合内存资源有限的系统

六. 应用

题目描述

[复制 Markdown](#) [展开](#) [进入 IDE 模式](#)

您需要写一种数据结构（可参考题目标题），来维护一个有序数列。

其中需要提供以下操作：翻转一个区间，例如原有序序列是 5 4 3 2 1，翻转区间是 $[2, 4]$ 的话，结果是 5 2 3 4 1。

输入格式

第一行两个正整数 n, m ，表示序列长度与操作个数。序列中第 i 项初始为 i 。
接下来 m 行，每行两个正整数 l, r ，表示翻转的区间。

输出格式

输出一行 n 个正整数，表示原始序列经过 m 次变换后的结果。

输入输出样例

输入 #1

复制

```
5 3
1 3
1 3
1 4
```

输出 #1

复制

```
4 3 2 1 5
```

说明/提示

【数据范围】

对于 100% 的数据， $1 \leq n, m \leq 100000$ ， $1 \leq l \leq r \leq n$ 。

六. 应用

分析:

1. 构建初始顺序序列并构建伸展树

首先我们使用伸展树来存储序列中的元素从1到n的初始序列。

2. 翻转操作的实现（懒标记和旋转操作）

- 翻转区间操作：每次对区间 $[L, R]$ 进行翻转时，我们可以通过伸展树的旋转操作将这个区间的元素旋转到树的根部，然后通过懒标记来标记该区间需要翻转，最终通过懒标记机制推迟实际的翻转。
- 懒标记（rev）：

每个节点都可以有一个懒标记 rev，当 $rev[x] == 1$ 时，表示该节点的子树需要进行反转。当访问一个节点时，我们首先检查其懒标记。如果标记为反转，我们就交换该节点的左右子树，并将懒标记传递给其子节点。这样可以避免每次翻转时都需要直接对节点进行处理，减少操作的复杂度。

3. 翻转操作的实现步骤

找到区间的根：首先，通过伸展树的查找操作，找到区间 $[L, R]$ 的起始节点和结束节点。

伸展操作：通过一系列的旋转操作，将这两个节点分别移动到树的根部。

反转子树：一旦区间节点被旋转到根部，我们就可以通过懒标记机制进行反转操作。懒

标记传播：将 rev 标记传递给子节点，以确保后续的操作会考虑到这个反转。

4. 查询和输出操作

在每次翻转操作完成之后，我们需要输出整个序列的当前状态。此时，通过伸展树的查询操作，我们可以获取当前顺序序列中每个元素的值。



六. 应用 代码

```
1 #include<bits/stdc++.h>
2 #define N 100005
3 using namespace std;
4 int n,m;
5 int fa[N],ch[N][2],size[N],rev[N],rt;
6 inline void pushup(int x){
7     size[x]=size[ch[x][0]]+size[ch[x][1]]+1; // 每个节点所在子树大小
8 }
9 void pushdown(int x){
10     if(rev[x]){
11         swap(ch[x][0],ch[x][1]);
12         rev[ch[x][0]]^=1; rev[ch[x][1]]^=1; rev[x]=0;
13     } // 如果节点x标记为反转 (rev[x]为1), 则交换左右子树, 并递归标记其子节点。
14 }
15 void rotate(int x,int &k){ // 进行一次旋转操作, 将节点x提升到其父节点的位置。
16     int y=fa[x],z=fa[y],kind;
17     if(ch[y][0]==x)kind=1;else kind=0;
18     if(y==k)k=x;
19     else{if(ch[z][0]==y)ch[z][0]=x;else ch[z][1]=x;}
20     ch[y][kind^1]=ch[x][kind];fa[ch[y][kind^1]]=y;
21     ch[x][kind]=y;fa[y]=x;fa[x]=z;
22     pushup(x);pushup(y);
23 }
24 void splay(int x,int &k){ // 伸展操作, 目的是将节点x旋转到树的根节点, 从而保证接下来的操作能高效进行
25     while(x!=k){
26         int y=fa[x],z=fa[y];
27         if(y!=k){
28             if((ch[y][0]==x)^(ch[z][0]==y))rotate(x,k);
29             else rotate(y,k);
30         }
31         rotate(x,k);
32     }
33 }
```


六. 应用 代码

```
34 void build(int l,int r,int f){//在给定区间内构建树
35     if(l>r)return;
36     int mid=(l+r)/2;
37     if(mid<f)ch[f][0]=mid;else ch[f][1]=mid;
38     fa[mid]=f;size[mid]=1;
39     if(l==r)return;
40     build(l,mid-1,mid);build(mid+1,r,mid);
41     pushup(mid);
42 }
43 int find(int x,int k){//在以x为根的子树中找到第k个节点
44     pushdown(x);int s=size[ch[x][0]];
45     if(k==s+1)return x;
46     if(k<=s)return find(ch[x][0],k);
47     else return find(ch[x][1],k-s-1);
48 }
49 void rever(int l,int r){//执行翻转操作。它首先通过find找到要翻转的区间，然后通过伸展操作将该区间翻转。
50     int x=find(rt,l),y=find(rt,r+2);
51     splay(x,rt);splay(y,ch[x][1]);int z=ch[y][0];
52     rev[z]^=1;
53 }
54 int main(){
55     cin>>n>>m;
56     rt=(n+3)/2;build(1,n+2,rt);
57     for(int i=1;i<=m;i++){
58         int L,R;
59         cin>>L>>R;
60         rever(L,R);
61     }
62     for(int i=2;i<=n+1;i++){
63         cout << find(rt, i) - 1 << " ";
64     }
65     return 0;
66 }
```

参考文献

- [1].Sleator, Daniel D., and Robert E. Tarjan. "Self-Adjusting Binary Search Trees." Journal of the ACM (JACM), vol. 32, no. 3, 1985, pp. 652-686.
- [2].<https://blog.csdn.net/u014634338/article/details/49586689> 2015-11-02
- [3].<https://www.luogu.com.cn/problem/P3391> 2018-04-25

Thanks!

